

Universidad Tecnológica de Querétaro

Ingeniería en Desarrollo y Gestión de Software

Seguridad en Desarrollo de Aplicaciones

Grupo: IDGS17

Reporte de Práctica

Análisis de Código Estático y Vulnerabilidades con SonarQube

Presenta:

Saul Noe Perez Aguilera

Santiago de Querétaro, Qro.

12 de junio de 2026

Cuestionario de la Práctica

1. ¿Qué diferencia hay entre un Security Hotspot y un Bug en SonarQube?

La diferencia principal radica en la certeza del problema y la necesidad de contexto humano:

- **Bugs:** Son errores lógicos comprobables en el código que rompen el funcionamiento esperado de la aplicación (por ejemplo, una condición que siempre evalúa a verdadero o un ciclo infinito). Requieren corrección inmediata.
- **Security Hotspots:** Son marcados como posibles vulnerabilidades de seguridad. Se trata de fragmentos de código sensibles que *podrían* ser peligrosos dependiendo del contexto, pero requieren que un desarrollador los revise manualmente para confirmar si realmente representan un riesgo o si la implementación actual es segura.

2. ¿Por qué es inseguro concatenar directamente el input del usuario en una query SQL? Explica el ataque con un ejemplo de payload.

Es inseguro porque el motor de la base de datos no puede distinguir entre las instrucciones legítimas del desarrollador y los datos proporcionados por el usuario. Al concatenar texto, un atacante puede inyectar caracteres especiales para alterar la estructura lógica de la consulta original.

Ejemplo de ataque (SQL Injection):

Imagina que la consulta en el código es:

```
string query = "SELECT * FROM Users WHERE Username = '" + input + "'";
```

Si el usuario ingresa el siguiente *payload* en el campo de texto: ' OR '1'='1

La consulta final que se ejecuta en la base de datos se transforma en:

```
SELECT * FROM Users WHERE Username = '' OR '1'='1'
```

Como 1=1 siempre es verdadero, la base de datos devolverá todos los registros de la tabla de usuarios, permitiendo al atacante eludir la autenticación o extraer información confidencial sin saber ninguna contraseña.

3. ¿Qué es el Code Coverage y por qué SonarQube lo considera relevante para la seguridad?

El Code Coverage (Cobertura de Código) es una métrica que indica el porcentaje de líneas de código fuente que fueron ejecutadas y validadas por las pruebas automatizadas (pruebas unitarias).

SonarQube lo considera crucial para la seguridad porque el código que no está cubierto por pruebas es un “punto ciego”. Si la lógica de validación de entradas, los algoritmos de encriptación o los manejadores de errores de autenticación no tienen pruebas, los desarrolladores no tienen forma automatizada de garantizar que esos controles de seguridad estén funcionando correctamente tras cada actualización del sistema.

4. Si un colega sube el token de SonarQube a GitHub por error, ¿qué pasos seguirías para mitigar el riesgo?

Se debe actuar de inmediato siguiendo un protocolo de respuesta a incidentes:

- a) **Revocar la credencial:** Entrar inmediatamente a la interfaz de SonarQube y eliminar el token comprometido para invalidarlo y evitar accesos no autorizados.
- b) **Auditoría rápida:** Revisar los logs de acceso de SonarQube para confirmar si el token fue utilizado por algún tercero mientras estuvo expuesto.
- c) **Generar un nuevo token:** Crear una nueva credencial y distribuirla de forma segura (por ejemplo, mediante un gestor de secretos).
- d) **Limpiar el historial de Git:** No basta con hacer un nuevo *commit* borrando el token, ya que quedaría en el historial. Se deben usar herramientas como `git filter-repo` o BFG Repo-Cleaner para purgar permanentemente ese dato del historial del repositorio público.